CHAPITRE 1

Fondamentaux de Java

- Historique et caractéristiques du langage
- Installation et environnement de développement
- Variables, types, opérateurs, structures de contrôle
- JDK, JVM, JRE et exécution d'un programme
- Organisation d'un projet Java/Maven
- Tableaux, méthodes et paramètres

1. HISTORIQUE

Historique et caractéristiques de Java

1

Origines (1995, Sun Microsystems)

Créé par James Gosling pour des systèmes embarqués ("Oak"), puis adapté au Web sous le nom Java. Racheté par Oracle en 2010.

2

"Write Once, Run Anywhere" (WORA)

Le code source est compilé en bytecode indépendant de la plateforme, exécuté par une JVM spécifique à chaque OS.

3

Langage orienté objet

Tout (ou presque) est objet ; syntaxe proche du C/C++ mais sans pointeurs explicites ni héritage multiple de classes.

4

Robustesse et portabilité

Typage fort, gestion automatique de la mémoire (Garbage Collector), gestion structurée des exceptions.

5

Écosystème riche

API standard très large, Maven/Gradle pour la gestion de projet, immense communauté et bibliothèques (Spring, Hibernate...).

6

Versions LTS

Java 8, 11, 17, 21 (**LTS**). Ce module utilise Java 27, la version recommandée en production en 2026.

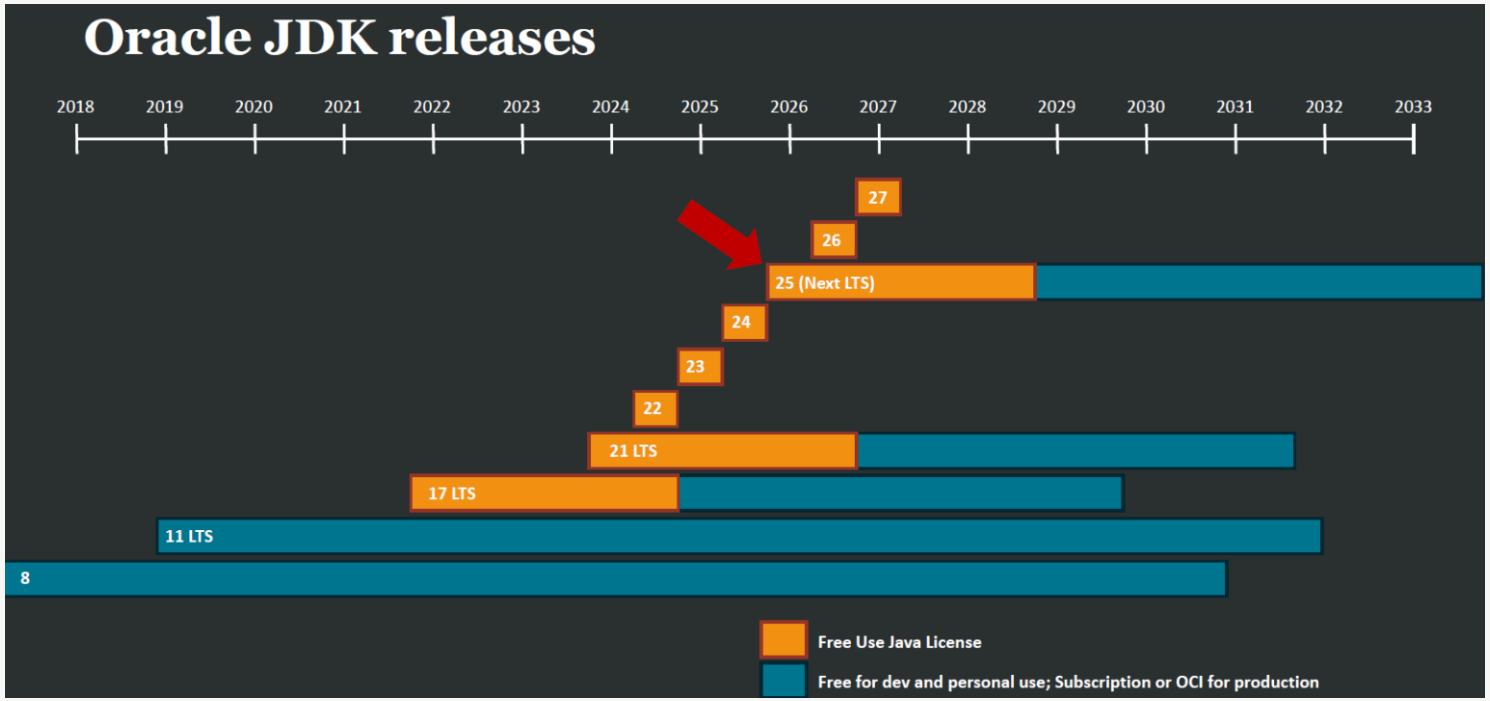
Historique de Java

1991	Début du projet "Green" chez Sun Microsystems par James Gosling
1995	Première version publique de Java (Java 1.0)
1996	Introduction de Java Applets pour le web
1998	Java 2 (J2SE 1.2) - Refonte majeure de la plateforme
2004	Java 5 - Ajout des génériques, annotations, autoboxing
2010	Oracle acquiert Sun Microsystems
2014	Java 8 - Expressions lambda, Stream API
2018	Nouveau cycle de release tous les 6 mois
2023	Java 21
2025	Java 25 (LTS)
2026	Java 27 now available, **September 16, 2026**

Logo de Sun Microsystems, créateur de Java



James Gosling, père fondateur de Java



Caractéristiques principales de Java

Orienté objet

Tout en Java est un objet, facilitant la modularité et la réutilisation du code

Portabilité (WORA)

Write Once, Run Anywhere - Le bytecode Java s'exécute sur n'importe quelle JVM

Garbage Collector

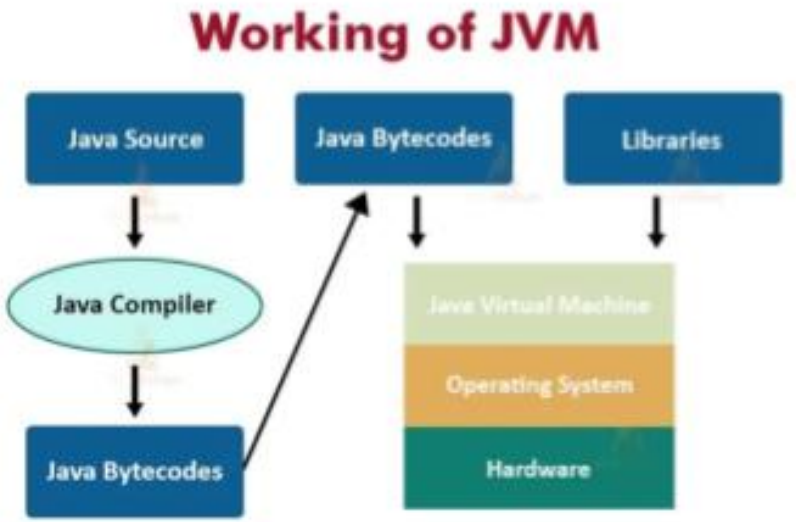
Gestion automatique de la mémoire, libérant le développeur de cette tâche

Sécurité intégrée

Architecture de sécurité robuste avec sandbox d'exécution et vérification du bytecode

Bibliothèque standard riche

API complète couvrant réseau, interface graphique, collections, concurrence, etc.



Architecture de la JVM (Java Virtual Machine) permettant la portabilité du code Java

Caractéristiques principales de Java







ORACLE

Java SE

Java mobile

J2ME platform

JavaFx

JAVA SWING

Java EE™

Caractéristiques principales de Java: Java SE

- ✓ Le terme « **Java** » fait bien évidemment référence à un langage.
- ✓ Mais également à une plate-forme : son nom complet est « **Java SE** » pour **Java Standard Edition**, et était anciennement raccourci « **J2SE** ».
- ✓ **Java SE** est constituée de nombreuses bibliothèques, ou **API**.
- ✓ Par exemple : *java.lang*, *java.io*, *java.math*, *java.util*, etc.
- ✓ **Java SE** regroupe toutes les bibliothèques qui contiennent un nombre important de classes et de méthodes prêtes à l'emploi pour effectuer toutes sortes de tâches et afin de développer les applications Desktop.



Caractéristiques principales de Java: Java EE (~~J2EE~~ ou ~~JEE~~)

- ✓ **Java EE** signifie **Java Enterprise Edition**.
- ✓ **Java EE** est à une extension de la plate-forme standard.
- ✓ La plate-forme **Java EE** est construite sur le langage Java et la plate-forme Java SE.
- ✓ L'objectif majeur de **Java EE** est de faciliter le développement d'applications web robustes et distribuées, déployées et exécutées sur un **serveur d'applications**.
- ✓ **Java EE 8** était la dernière version de **Java EE** d'Oracle, et maintenant, Oracle a déplacé **Java EE** vers une fondation open source (*Eclipse Foundation*), et elle s'appelle **Jakarta EE**.





1. HISTORIQUE

Java face à C / C++

C / C++ (déjà connu)

- Compilation directe en code machine natif
- Gestion manuelle de la mémoire (malloc/free, new/delete)
- Pointeurs explicites et arithmétique de pointeurs
- Héritage multiple de classes possible
- Portabilité limitée : à recompiler par plateforme
- Pas de gestion structurée d'exceptions en C

Java (nouveauté du module)

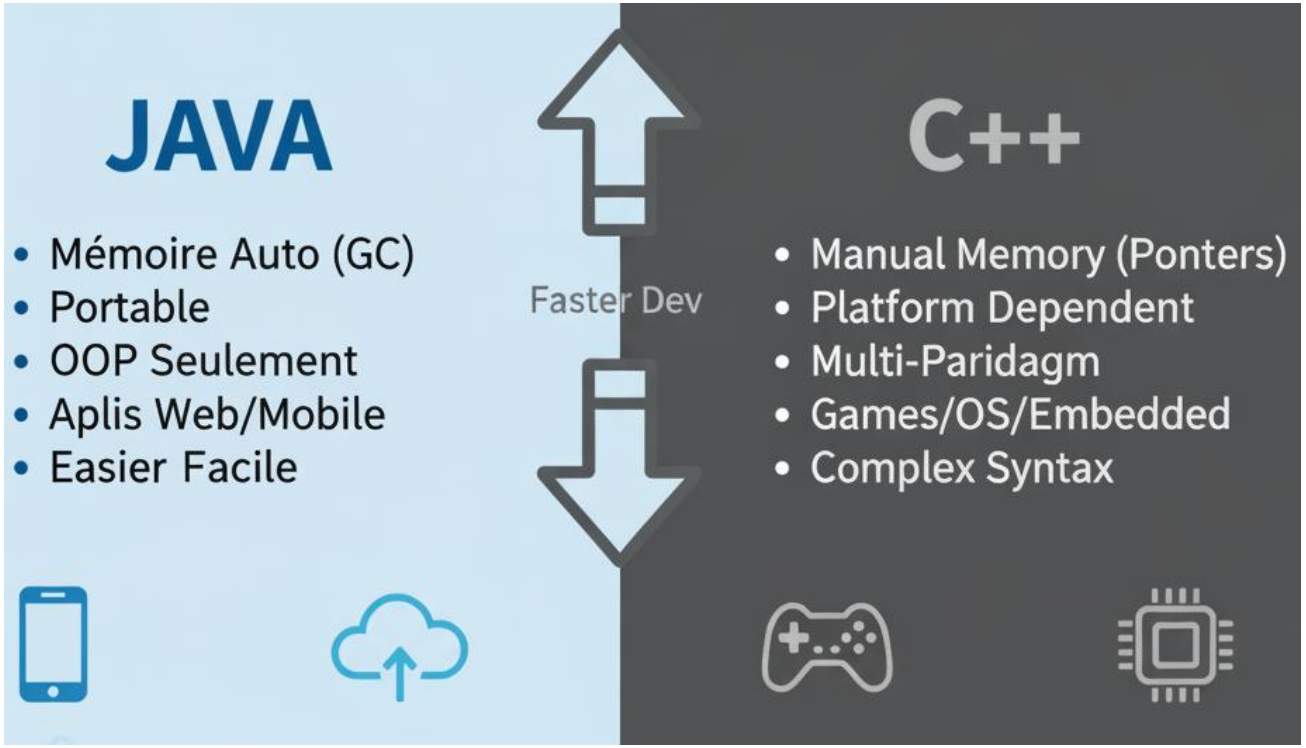
- Compilation en bytecode + exécution par la JVM
- Gestion automatique de la mémoire (Garbage Collector)
- Pas de pointeurs explicites : références sécurisées
- Héritage simple de classes + interfaces multiples
- Portabilité totale : "compilé une fois, exécuté partout"
- Gestion des exceptions obligatoire pour les erreurs vérifiées

1. HISTORIQUE

Java face à C / C++

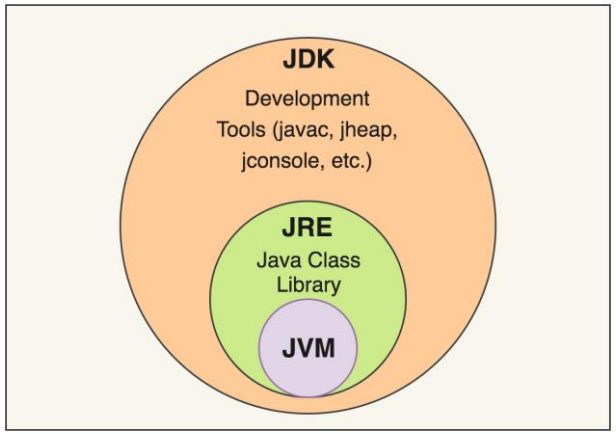
Caractéristique	Java	C/C++
Gestion mémoire	Automatique (Garbage Collector)	Manuelle (malloc/free, new/delete)
Pointeurs	Références uniquement	Pointeurs explicites
Héritage	Simple uniquement	Simple et multiple
Portabilité	Élevée (WORA)	Dépendante de la plateforme
Compilation	Bytecode → JVM	Code machine natif





JDK, JRE et JVM : bien distinguer

- ✓ JDK (Java Development Kit) : Fournit outils pour compiler, déboguer, exécuter le code Java.
- ✓ JRE (Java Runtime Environment) : Environnement nécessaire pour exécuter les applications Java.
- ✓ JVM (Java Virtual Machine) : Simule l'exécution des programmes Java.



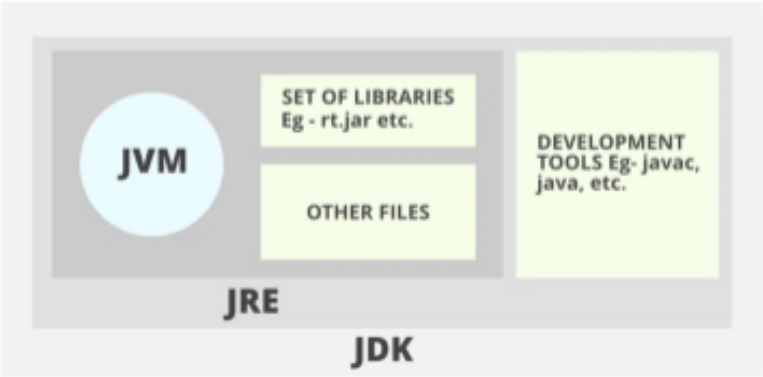
Composant	Contient	Rôle
JDK	JRE + compilateur (javac) + outils (javadoc, jar, jdb…)	Nécessaire pour développer et compiler du Java
JRE	JVM + bibliothèques standard (java.lang, java.util…)	Nécessaire pour exécuter une application Java
JVM	Interpréteur bytecode + JIT + Garbage Collector	Exécute le bytecode .class de façon portable

JDK, JRE et JVM : bien distinguer

JDK (Java Development Kit)

Environnement complet pour le développement d'applications Java

- Compilateur Java (javac)
- Outils de développement (javadoc, jar, etc.)
- Débogueur (jdb)
- Inclut le JRE



Relation entre JDK, JRE et JVM - Structure en couches

JRE (Java Runtime Environment)

Environnement d'exécution pour les applications Java

- Bibliothèques de classes Java (API)
- Chargeur de classes
- Inclut la JVM

JVM (Java Virtual Machine)

Machine virtuelle qui exécute le bytecode Java

- Interprète le bytecode
- Compilateur JIT (Just-In-Time)
- Garbage Collector
- Gestionnaire de mémoire

L'environnement de développement Java (IDE)

Ligne de commande

javac

Compilateur Java qui transforme le code source (.java) en bytecode (.class)

java

Lanceur de la JVM qui exécute le bytecode Java

jar

Outil pour créer et gérer des archives JAR (Java ARchive)

jdb

Débogueur Java en ligne de commande

Outils de build

Maven

Gestion de dépendances et automatisation de build

Gradle

Système de build flexible basé sur Groovy/Kotlin

Environnements de Développement Intégrés (IDE)

Eclipse IDE

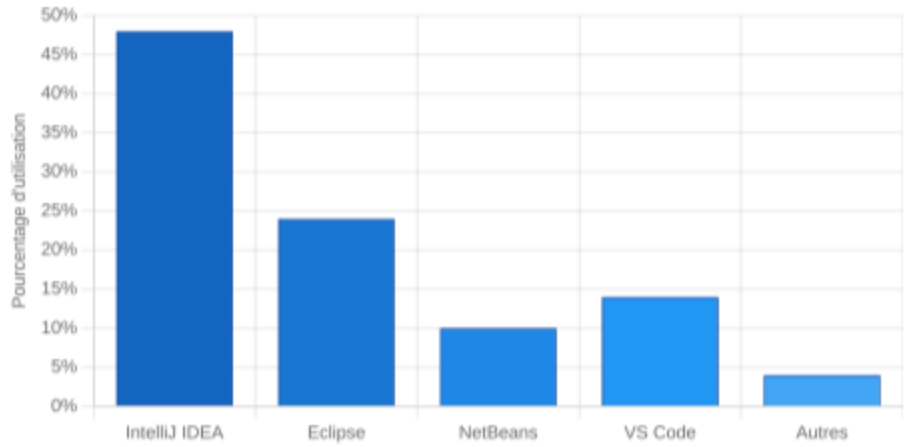
Open source, extensible via plugins

Apache NetBeans

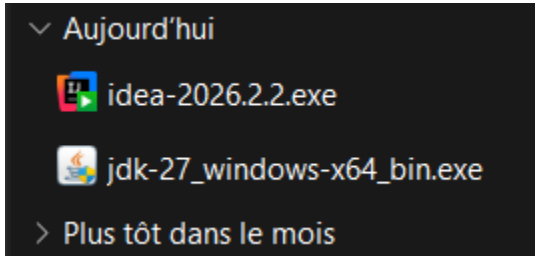
Modulaire, multi-langage

IntelliJ IDEA

Commercial, très puissant



L'environnement de développement Java (IDE)



Pour ce module : *JDK 27 LTS installé* + *IntelliJ IDEA comme IDE 2026,2,2* *(alternatives possibles : Eclipse, NetBeans).*



L'environnement de développement Java (IDE)

- ✓ **IDE** - Integrated Development Environment - Environnement de développement intégré; un IDE n'est pas un éditeur de texte.

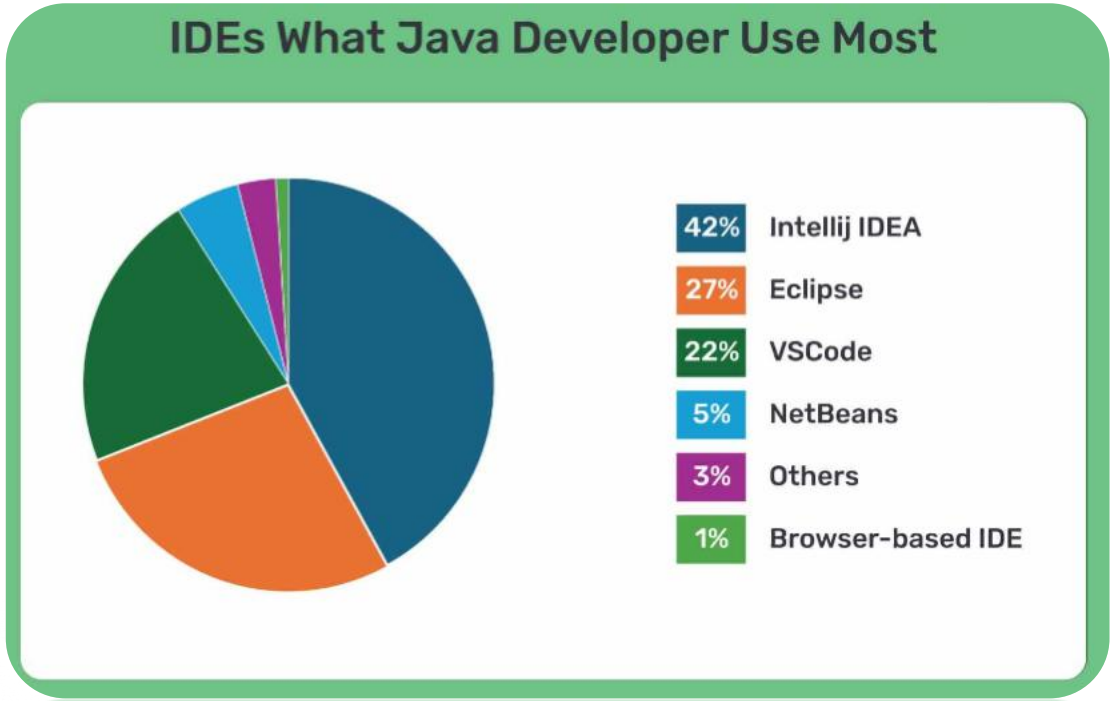
- ✓ **IDE** a un ensemble de fonctionnalités :
  - La vérification de la syntaxe du langage ;
  - La détection des erreurs ;
  - L'auto-complétion ;
  - Un compilateur ;
  - Un serveur.

- ✓ Principaux **IDEs** : Eclipse, IntelliJ IDEA, NetBeans.



L'environnement de développement Java (IDE)

Apache NetBeans IDE

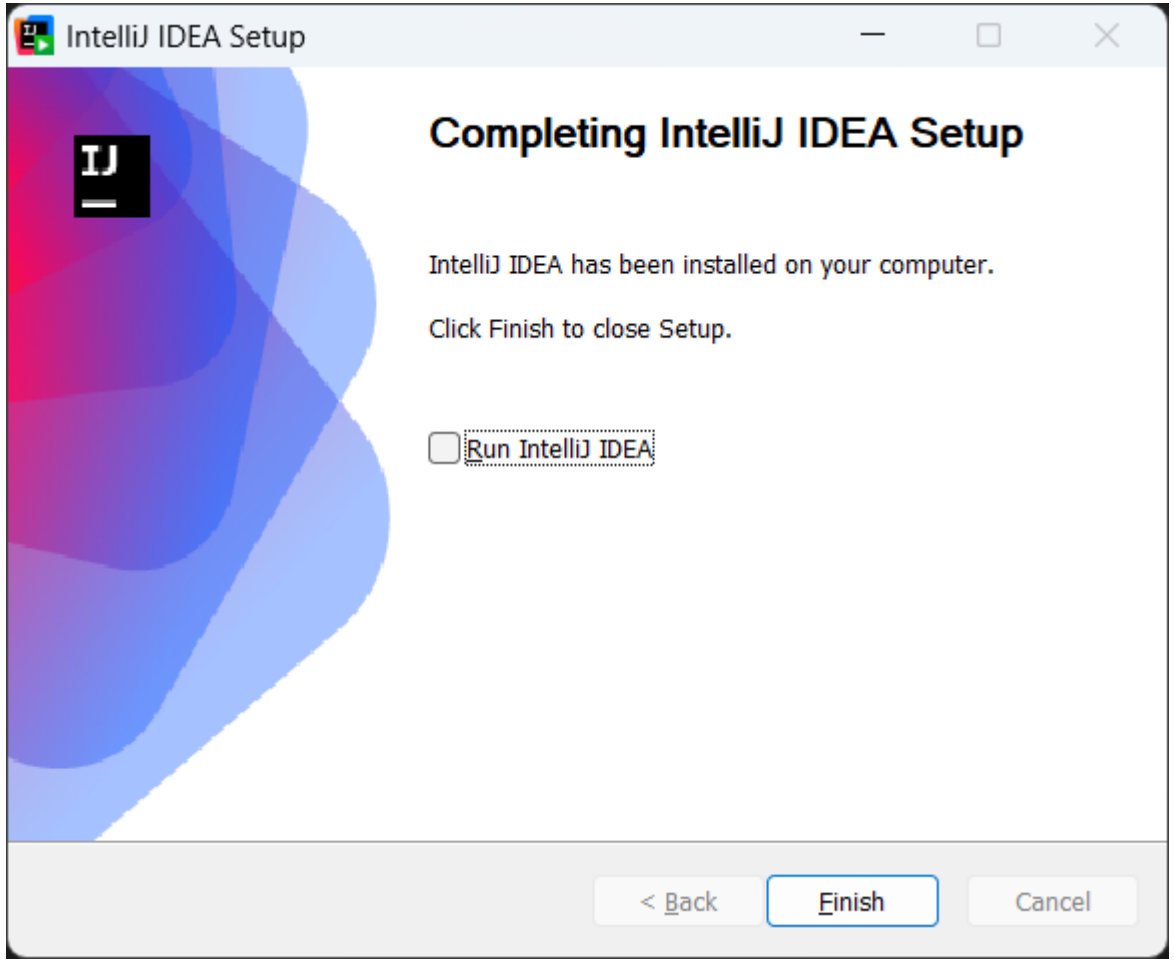


eclipse

Visual Studio Code

L'environnement de développement Java (IDE)

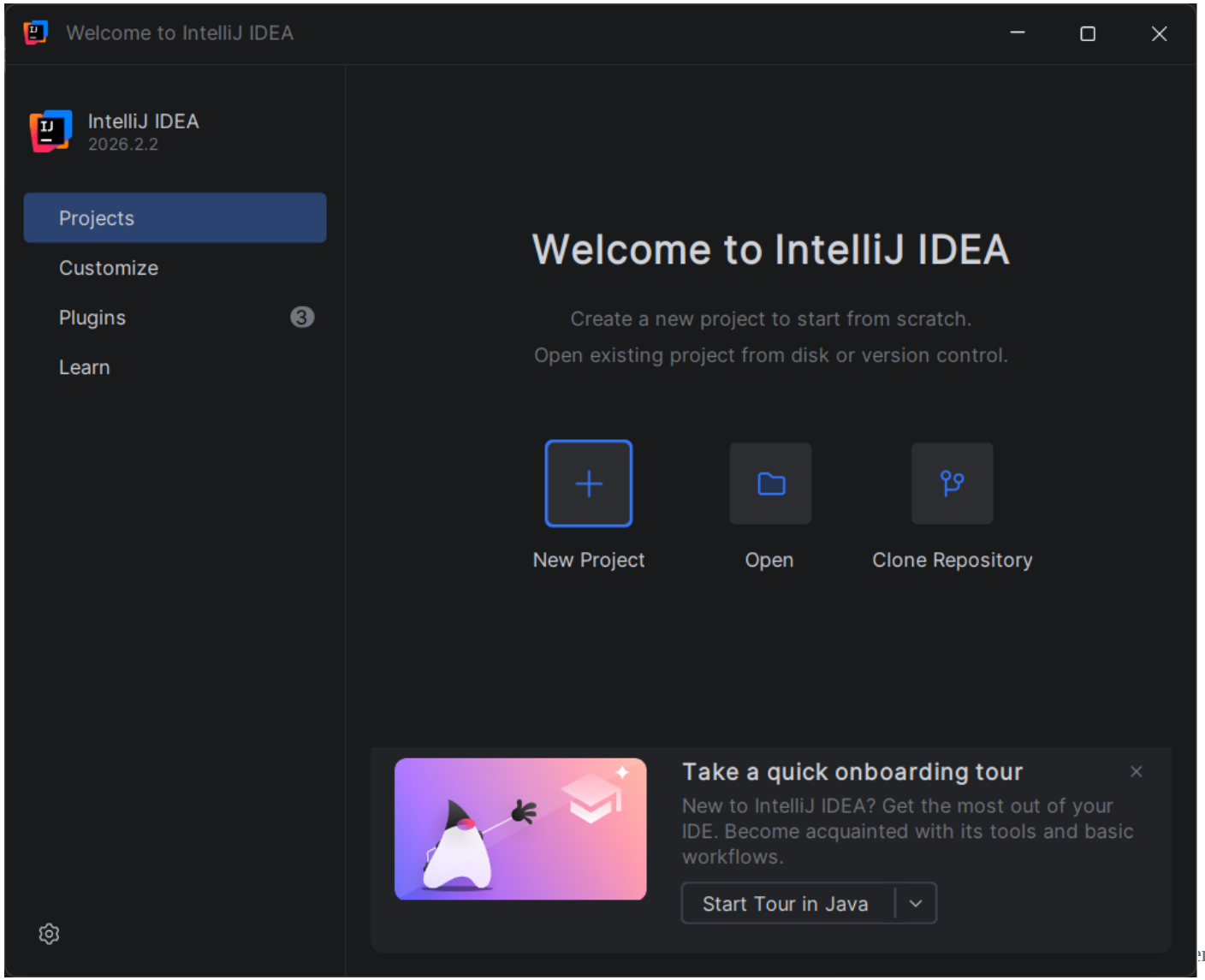




2. JDK / JVM

L'environnement de développement Java (IDE)





2. JDK / JVM

Du code source à l'exécution

1

.java (source)

Fichier texte contenant le code source Java, écrit par le développeur.

2

javac (JDK)

Le compilateur Java traduit le code source en bytecode indépendant de la plateforme.

3

.class (bytecode)

Instructions bytecode portables, stockées dans des fichiers .class (ou un .jar).

4

JVM (java)

La machine virtuelle Java interprète / compile à la volée (JIT) le bytecode pour le processeur cible.

5

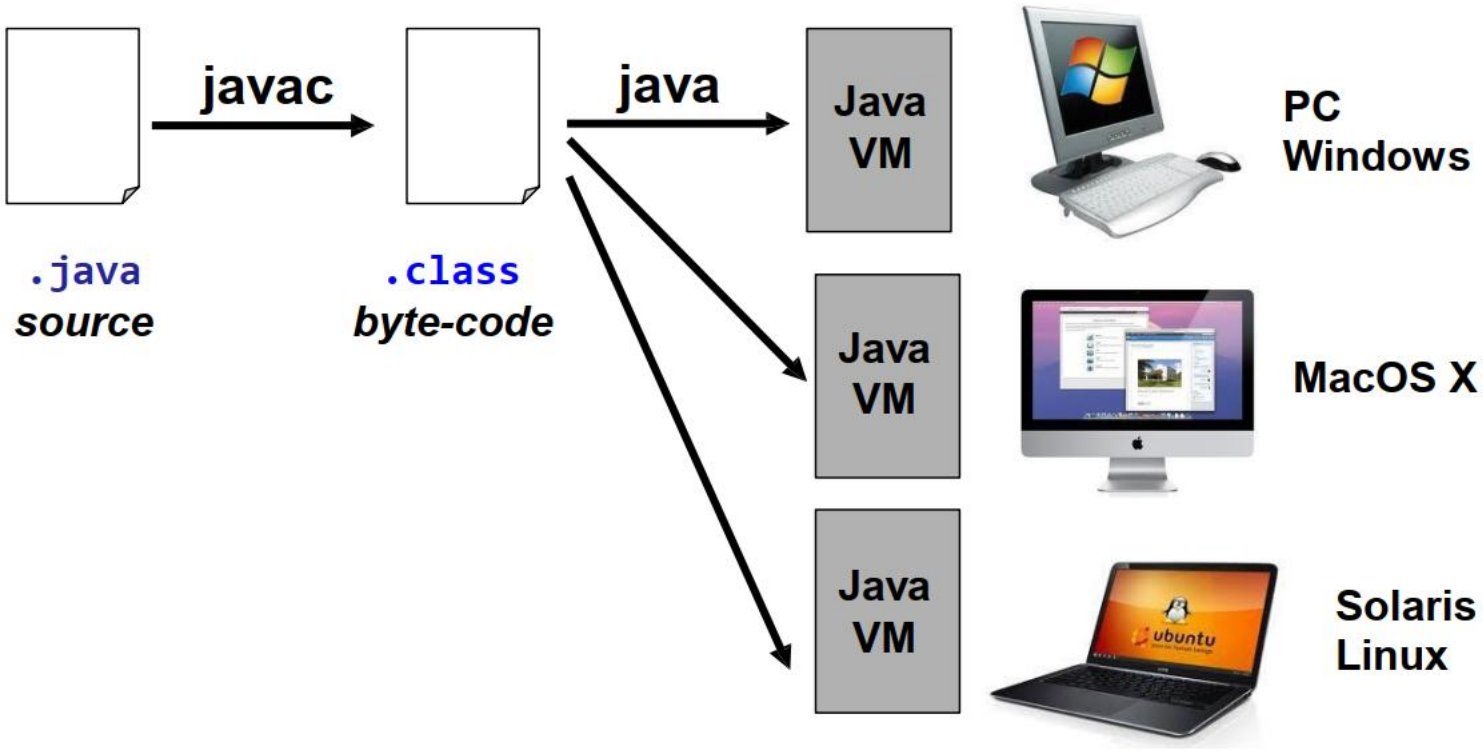
Exécution

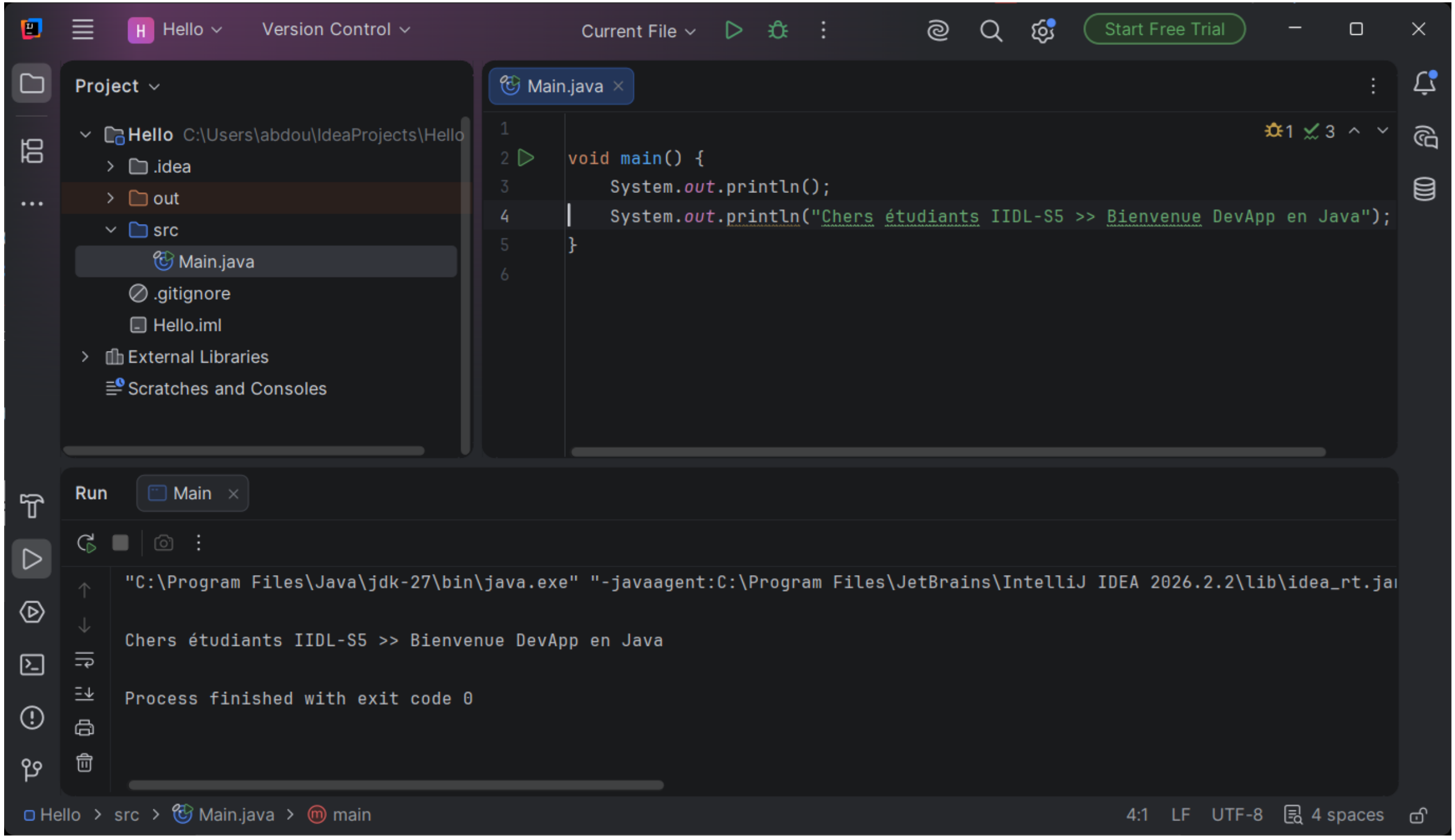
Le programme s'exécute avec accès au Garbage Collector, à la sécurité et aux threads de la JVM.

JDK (Java Development Kit) = JRE (environnement d'exécution) + outils de développement (javac, javadoc, jar…). La JVM (Java Virtual Machine) est le composant qui exécute le bytecode : elle rend Java portable sur Windows, Linux, macOS.

Du code source à l'exécution

- **byte-code assure la portabilité des programmes Java**
  - langage d'une Machine Virtuelle
  - à l'exécution un interpréteur simule cette machine virtuelle




Hello
Version Control
Current File
Start Free Trial
Project
Hello C:\Users\abdou\IdeaProjects\Hello
.idea
out
src
Main.java
.gitignore
Hello.iml
External Libraries
Scratches and Consoles
Main.java
1
2 void main() {
3 System.out.println();
4 System.out.println("Chers étudiants IIDL-S5 >> Bienvenue DevApp en Java");
5 }
6
1 3
Run
Main
"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.2\lib\idea_rt.ja
Chers étudiants IIDL-S5 >> Bienvenue DevApp en Java
Process finished with exit code 0
Hello > src > Main.java > main
4:1
LF
UTF-8
4 spaces

3. PREMIER PROGRAMME

Structure d'un programme Java

HelloWorld.java

```
1  // Un programme Java minimal
2  public class  HelloWorld {
3  // La méthode principale
4  public static void   main(String[] args) {
5   System.out.println( "Bonjour, Java !"  );
6
7  }
   }
```

Compilation et exécution

```
1  $ javac HelloWorld.java
2  $ java HelloWorld
3  Bonjour, Java !
```

Classe publique

Le nom du fichier doit correspondre au nom de la classe publique

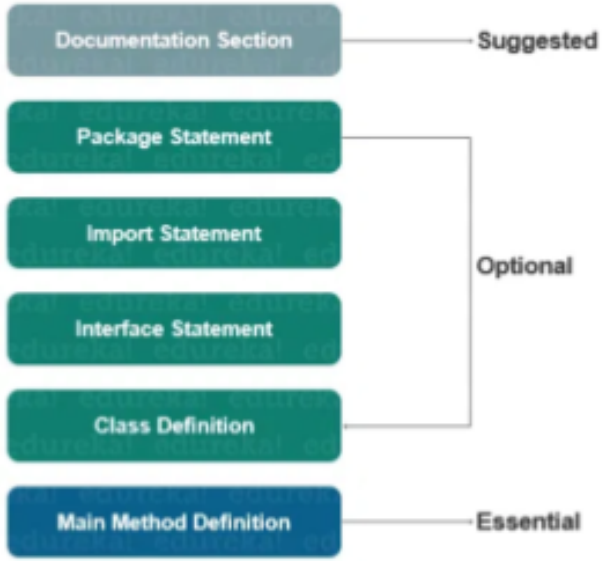
Méthode main

Point d'entrée obligatoire:

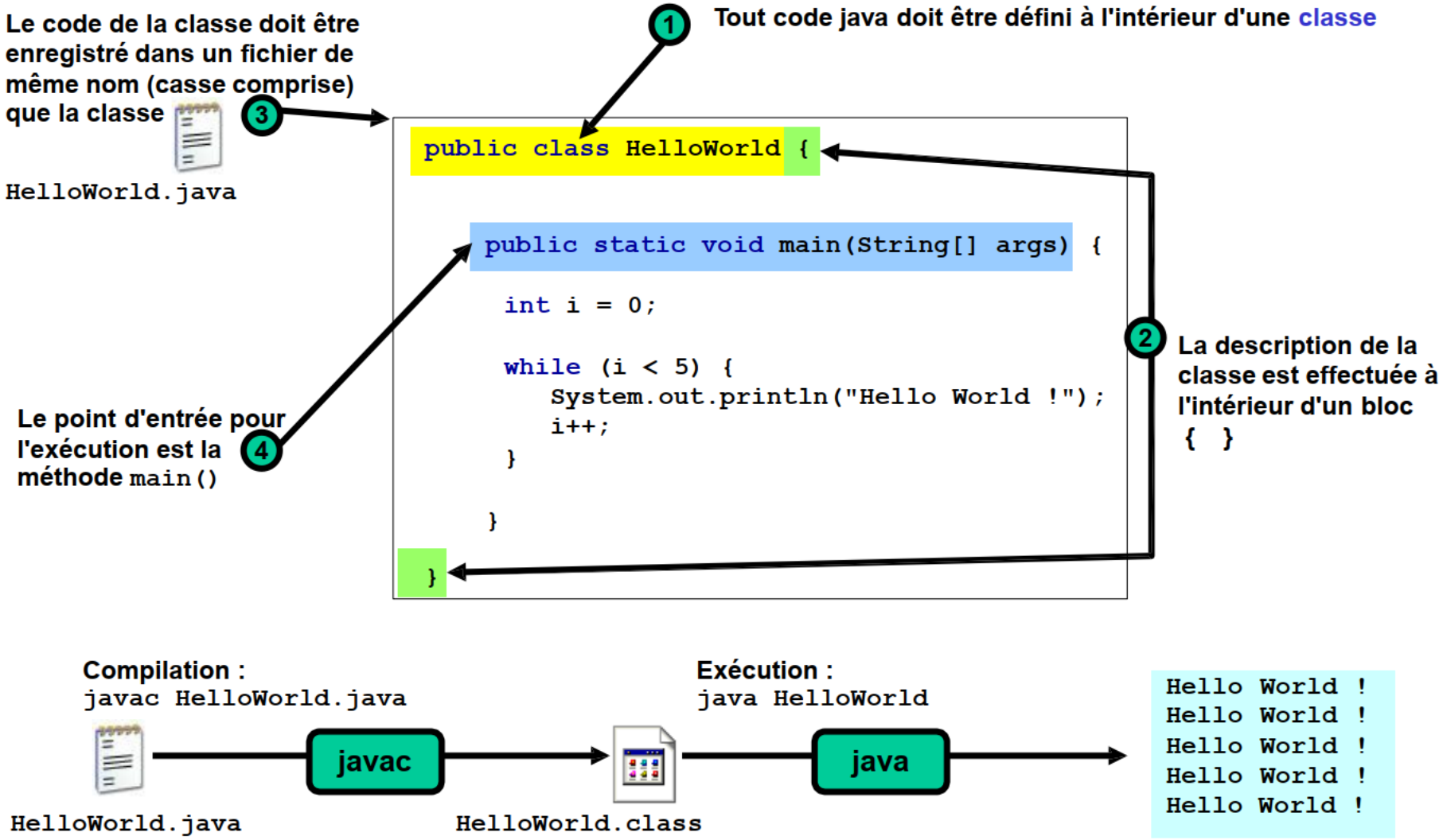
public static void main(String[] args)

Sortie standard

System.out.println() affiche du texte dans la console

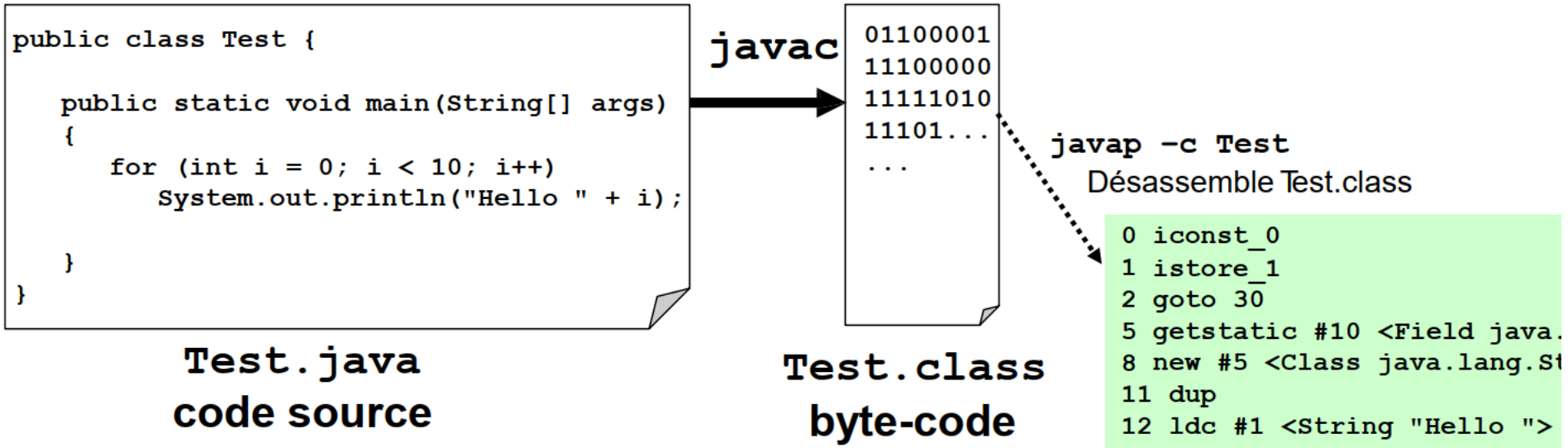


Structure d'un programme Java



Structure d'un programme Java

- **Compilation d'un programme JAVA : génération de byte-code**



- **Le byte-code est :**
  - proche d'un langage machine
  - indépendant de la plateforme d'exécution (matériel + OS)

Structure d'un programme Java

- Structure de base :

public class **NomDeLaClasse** {
  public static void **main**(String[] args) {
    // code ici
  }
}

•**Explication des éléments** : **Classe** : Conteneur de code.

•**Méthode main** : Point d'entrée de l'application.

•**System.out.println** : Affiche du texte.

- **Exemple :** Mon premier programme en Java, affiche '**Hello, World!**'

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

3. PREMIER PROGRAMME

Structure d'un programme Java

```
// Fichier : Main.java
package ma.uae.gestion;

public class Main {

    public static void main(String[] args) {
        System.out.println("Bonjour, Java !");
    }

}
```

Éléments clés

- package : espace de noms qui organise les classes du projet
- public class Main : une classe publique par fichier, du même nom que le fichier
- public static void main(String[] args) : point d'entrée obligatoire, appelé par la JVM
- static : la méthode main s'exécute sans créer d'objet Main
- System.out.println() : affiche une ligne sur la sortie standard (console)
- Chaque instruction se termine par un point-virgule ;

5. VARIABLES ET TYPES

Variables, constantes et types primitifs

1

8 types primitifs

byte, short, int, long (entiers) · float, double (décimaux) · char (caractère) · boolean (vrai/faux).

2

Déclaration et affectation

int age = 20; double moyenne = 12.5; boolean valide = true; — le type est fixé à la compilation (typage statique).

3

Constantes avec final

final double TAUX_TVA = 0.20; — une constante ne peut plus être réaffectée après son initialisation.

4

Types référence vs primitifs

String, tableaux et objets sont des types référence : la variable contient une adresse mémoire, pas la donnée elle-même.

5

Conversion de types (casting)

Conversion implicite (int → double) sans perte, ou explicite (double) x pour une conversion avec perte possible.

6

Emballage (Wrapper classes)

Integer, Double, Boolean… permettent d'utiliser les primitifs comme des objets (utile pour les collections).

5. VARIABLES ET TYPES

Les 8 types primitifs Java

Type	Taille	Plage / usage	Exemple
`byte`	8 bits	-128 à 127	byte b = 10;
`short`	16 bits	-32 768 à 32 767	short s = 1000;
`int`	32 bits	entier standard	int age = 21;
`long`	64 bits	grands entiers (suffixe L)	long id = 100000L;
`float`	32 bits	décimal simple précision (suffixe f)	float pi = 3.14f;
`double`	64 bits	décimal double précision (défaut)	double moy = 12.5;
`char`	16 bits	un caractère Unicode	char c = 'A';
`boolean`	1 bit (JVM)	true / false	boolean ok = true;

6. OPÉRATEURS

Opérateurs et expressions

```java
int a = 7, b = 2;
System.out.println(a + b);    // 9
System.out.println(a % b);    // 1 (modulo)
System.out.println(a / b);    // 3 (division entière !)
System.out.println((double) a / b); // 3.5

boolean estMajeur = (a >= 18) && (b > 0);
int c = 5;
c += 3;    // c = c + 3  -> 8
c++;       // post-incrémentation -> 9

int note = 12;
String mention = (note >= 10) ? "Admis" : "Ajourné";
```

Familles d'opérateurs

- Arithmétiques : + - * / % (attention : division entre int tronque le résultat)
- Relationnels : == != > < >= <= (comparaison, retournent un boolean)
- Logiques : && || ! (évaluation court-circuit pour && et ||)
- Affectation composée : += -= *= /= %=
- Incrémentation : ++ -- (préfixe vs postfixe)
- Opérateur ternaire : condition ? valeurSiVrai : valeurSiFaux
- **final** = constante

Sorties en Java

Afficher un message sur l'écran :

- ✓ Pour afficher un message sur l'écran on utilise la méthode **print** ou **println** de la classe **System**.
- ✓ **Println** permet d'afficher le message sur console et retourner à la ligne.

La syntaxe :

```
System.out.println("Veuillez saisir un numero: ");
```

Sorties standard

→ **System.out.println()**
Affiche du texte avec un retour à la ligne

→ **System.out.print()**
Affiche du texte sans retour à la ligne

→ **System.out.printf()**
Affiche du texte formaté (similaire à printf en C)

```
// Exemples de sorties
System.out.println( "Bonjour" );
System.out.print( "Sans retour à la ligne " );
System.out.printf( "Nombre: %d, Texte: %s" , 42, "Java" );
```

7. ENTRÉES / SORTIES

Lire une saisie clavier (Scanner)

```
import java.util.Scanner;

public class Saisie {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Nom de l'étudiant : ");
        String nom = sc.nextLine();

        System.out.print("Note obtenue : ");
        double note = sc.nextDouble();

        System.out.printf("%s a obtenu %.2f/20%n", nom,
note);
        sc.close();
    }
}
```

Points d'attention

- Scanner(System.in) : lit les entrées depuis le clavier (console)
- nextLine() (chaîne), nextInt(), nextDouble()… : une méthode par type attendu
- Piège classique : mélanger nextInt() et nextLine() décale la saisie suivante
- System.out.printf() : affichage formaté façon C (%s, %d, %.2f, %n)
- Toujours fermer le Scanner (sc.close()) une fois terminé

Méthode	Type lu	Description
nextLine()	String	Lit une ligne entière
next()	String	Lit un mot
nextInt()	int	Lit un entier
nextDouble()	double	Lit un nombre à virgule
nextBoolean()	boolean	Lit un booléen

Différences avec C

C	Java
printf()	System.out.printf()
scanf()	Scanner.nextXXX()
Gestion manuelle des buffers	Gestion automatique
Risques de débordement	Sécurité intégrée

TP 1 : Calcul du Prix Brut

```java
import java.util.Scanner;

public class TP1 {
    Run | Debug
    public static void main(String[] args) {
        float PN, PB;
        final int TVA = 20;
        System.out.println(x:"Donner le PrixNet : ");
        Scanner sc=new Scanner(System.in);
        PN = sc.nextFloat();
        PB = PN + (PN*TVA/100);
        System.out.println(x:"Le PrixBrut : ");
        System.out.println(PB);
    }
}
```

- Écrire programme qui calcule le *Prix Brut* (type *float*) d'un article à partir du *Prix Net* (type *float*) et du pourcentage de *TVA (20 %)*, en utilisant la formule suivante :

$$PBRUT = PNET + PNET \cdot \frac{TVA}{100}$$

```
Donner le PrixNet :
100
Le PrixBrut :
120.0
PS C:\Users\User>
```

8. CONDITIONS

Structures conditionnelles

```java
int note = 14;

if (note >= 16) {
    System.out.println("Très bien");
} else if (note >= 10) {
    System.out.println("Admis");
} else {
    System.out.println("Ajourné");
}

// switch classique
switch (jour) {
    case 1 -> System.out.println("Lundi");
    case 2 -> System.out.println("Mardi");
    default -> System.out.println("Autre jour");
}
```

À retenir

- **if / else if / else** : blocs exécutés selon une condition booléenne
- Syntaxe moderne switch (flèche ->) : plus de break oublié possible
- Toujours prévoir un cas default pour les valeurs non gérées
- Une condition Java doit être un boolean (pas d'entier implicite comme en C)
- **switch** : alternative lisible pour de nombreux cas sur une même variable

L'instruction conditionnelle if

La syntaxe de l'instruction **if** peut être décrite de la façon suivante:

```
if (expression) instruction;
ou :
if (expression) {
   instruction1;
   instruction2;
   }
```

L'instruction conditionnelle else

```
if (expression) {
instruction1;
}
else {
instruction2;
}
```

L'instruction switch

Syntaxe :

```
switch( variable) {
 case valeur1: instr1;break;
 case valeur2: instr2;break;
 case valeurN: instrN;break;
 default: instr;break;
}
```

TP 2 : Maximum entre A et B

- Écrire un programme JAVA qui lit deux valeurs entières (A et B) au clavier et qui affiche la plus grande des deux valeurs.

```
Entrez la première valeur (A) : 5
Entrez la deuxième valeur (B) : 10
La plus grande valeur est : 10
```

```java
import java.util.Scanner;

public class TP2MaxAB {
    Run | Debug
    public static void main(String[] args) {
        // Créer un objet Scanner pour lire les entrées de l'utilisateur
        Scanner scanner = new Scanner(System.in);

        // Demander à l'utilisateur de saisir les deux valeurs
        System.out.print(s:"Entrez la première valeur (A) : ");
        int A = scanner.nextInt();

        System.out.print(s:"Entrez la deuxième valeur (B) : ");
        int B = scanner.nextInt();

        // Comparer les deux valeurs et afficher la plus grande
        if (A > B) {
            System.out.println("La plus grande valeur est : " + A);
        } else if (B > A) {
            System.out.println("La plus grande valeur est : " + B);
        } else {
            System.out.println(x:"Les deux valeurs sont égales.");
        }

        // Fermer le scanner
        scanner.close();
    }
}
```

TP 3 : Calculateur

- Ecrire un programme JAVA qui permet de saisir un expression arithmétique composé :
  - ✓ Deux entiers **A** et **B ,**
  - ✓ Une opération **OP** (' + ', ' - ', ' * ' ou ' / ').
- Puis, le programme affiche le résultat selon l'opération demandée.

```
Entrez le premier entier (A) : 9
Entrez le deuxième entier (B) : 5
Entrez l'opération ( + , - , * , / ) : *
Résultat : 45
```

```
Entrez le premier entier (A) : 10
Entrez le deuxième entier (B) : 5
Entrez l'opération ( + , - , * , / ) : /
Résultat : 2
```

```
Entrez le premier entier (A) : 5
Entrez le deuxième entier (B) : 0
Entrez l'opération ( + , - , * , / ) : /
Erreur : Division par zéro impossible.
```

```
Entrez le premier entier (A) : 13
Entrez le deuxième entier (B) : 4
Entrez l'opération ( + , - , * , / ) : ?
Opération invalide.
```

```java
public class CalculatriceSimple {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print(s:"Entrez le premier entier (A) : ");
        int A = scanner.nextInt();
        System.out.print(s:"Entrez le deuxième entier (B) : ");
        int B = scanner.nextInt();
        System.out.print(s:"Entrez l'opération ( + , - , * , / ) : ");
        char op = scanner.next().charAt(index:0); // Lecture de l'opération
        switch (op) {
            case '+':
                System.out.println("Résultat : " + (A + B));
                break;
            case '-':
                System.out.println("Résultat : " + (A - B));
                break;
            case '*':
                System.out.println("Résultat : " + (A * B));
                break;
            case '/':
                if (B != 0) {
                    System.out.println("Résultat : " + (A / B));
                } else {
                    System.out.println(x:"Erreur : Division par zéro impossible.");
                }
                break;
            default:
                System.out.println(x:"Opération invalide.");
        }
    }
```

9. BOUCLES

Structures répétitives

```java
// for : nombre d'itérations connu
for (int i = 1; i <= 5; i++) {
    System.out.println("Itération " + i);
}

// while : condition testée avant
int n = 10;
while (n > 0) {
    n -= 2;
}

// do-while : condition testée après (au moins 1 tour)
int reponse;
do {
    reponse = sc.nextInt();
} while (reponse != 0);
```

Choisir la bonne boucle

- **for** : nombre d'itérations connu à l'avance (parcours de tableau, compteur)
- **while** : condition évaluée avant chaque tour, peut ne jamais s'exécuter
- **do-while** : le corps s'exécute au moins une fois (ex : menu interactif)
- **for-each** (vu au CH3) : parcours simplifié des tableaux et collections
- **break** sort de la boucle, **continue** passe à l'itération suivante

TP 4 : Calcul de la factorielle de N (N!)

- Écrire un programme JAVA qui permet de calculer la factorielle **N! = 1 * 2 * 3 * ...* (N-1) * N**, d'un entier N en respectant que **0!=1**.

```
Entrez un entier N : 5
La factorielle de 5 est : 120
```

```
Entrez un entier N : 0
La factorielle de 0 est : 1
```

```
Entrez un entier N : -5
Erreur : la factorielle n'est définie pour N<0
```

```java
public class TP4Factoriel {
    public static void main(String[] args) {
        // Créer un objet Scanner pour lire l'entrée de l'utilisateur
        Scanner scanner = new Scanner(System.in);

        // Demander à l'utilisateur de saisir un entier N
        System.out.print(s:"Entrez un entier N : ");
        int N = scanner.nextInt();

        // Calculer la factorielle de N
        // Utiliser long pour éviter les débordements pour des grandes valeurs de N
        long resultat = 1;

        // Si N est un entier positif ou nul, calculer la factorielle
        if (N >= 0) {
            for (int i = 1; i <= N; i++) {
                resultat *= i;
            }
            System.out.println("La factorielle de " + N + " est : " + resultat);
        } else {
            // Si N est négatif, afficher un message d'erreur
            System.out.println(x:"Erreur : la factorielle n'est définie pour N<0");
        }

        // Fermer le scanner
        scanner.close();
    }
}
```

9. BOUCLES

TP 5 : Gestion des notes

- Écrire un programme JAVA qui permet de **saisir les notes** pour une classe de **05** étudiants, puis affiche la **moyenne** des notes ?

```
Entrez la note de l'étudiant 1: 13
Entrez la note de l'étudiant 2: 12
Entrez la note de l'étudiant 3: 11
Entrez la note de l'étudiant 4: 16
Entrez la note de l'étudiant 5: 15
La moyenne des notes est : 13.4
```

```java
import java.util.Scanner;
public class TP5Notes {
    Run | Debug
    public static void main(String[] args) {
        // Créer un objet Scanner pour lire l'entrée de l'utilisateur
        Scanner scanner = new Scanner(System.in);
        // Déclaration de la variable pour stocker la somme des notes
        double somme = 0;
        int nombreEtudiants = 05;
        // Saisie des notes pour les 20 étudiants
        for (int i = 1; i <= nombreEtudiants; i++) {
            System.out.print("Entrez la note de l'étudiant " + i + ": ");
            double note = scanner.nextDouble();
            somme += note; // Ajouter la note à la somme totale
        }
        // Calcul de la moyenne
        double moyenne = somme / nombreEtudiants;
        // Affichage de la moyenne
        System.out.println("La moyenne des notes est : " + moyenne);
        // Fermer le scanner
        scanner.close();
    }
}
```

Tableaux à une et deux dimensions

Les tableaux permettent de stocker plusieurs valeurs de même type dans une variable.

- Les valeurs contenues dans la variable sont repérées par un indice
- En langage java, les tableaux sont des objets

- On peut définir un tableau sur tous les types java.
- Syntaxe de définition d'un tableau :

type [ ] nom;

Déclaration :

- int tab [ ];
  String chaines[ ];

Exemple : Création d'un tableau

- tab = new int [20]; // tableau de 20 int
- chaines = new String [100]; // tableau de 100 chaine

- Le nombre d'éléments du tableau est mémorisé. Java peut ainsi détecter à l'exécution le dépassement d'indice et générer une exception. Mot clé « ***length*** »
  - Il est récupérable par : nomTableau.**length**

```
int taille = tab.length; //taille vaut 20
```

- Exemple :

```
int [ ] tab = new int[50];
System.out.println("Taille du tableau " + tab.length );
```

- Comme en C/C++, les indices d'un tableau commencent à '**0**'.

Initialisation :

```
tab[0]=1;
tab[1]=2; //etc.
noms[0] = new String( "Boule");
noms[1] = new String( "Bill");//etc
```

Création et initialisation simultanées :

```
String noms [ ] = {"Boule","Bill"};
Point  pts[ ] = { new Point (0, 0), new Point (10, -1)};
```

Tableaux à une et deux dimensions

Tab1.java

```
public class Tab1
{
  public static void main (String args[])
  {
    int tab[ ] ;
    tab = new int[4];
    tab[0]=5;
    tab[1]=3;
    tab[2]=7;
    tab[3]=tab[0]+tab[1];
  }
}
```

Pour déclarer une variable tableau on indique le *type* des éléments du tableau et le *nom de la variable tableau* suivi de *[ ]*

on utilise new <type> [taille]; pour initialiser le tableau

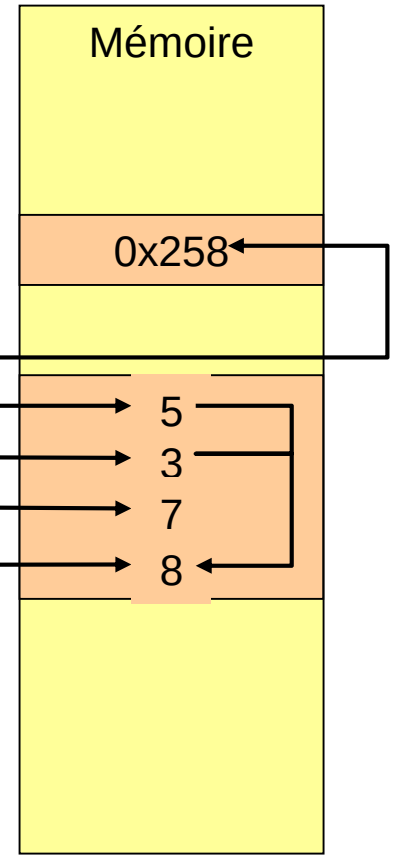
On peut ensuite affecter des valeurs au différentes cases du tableau : <nom_tableau>[indice]

Les indices vont toujours de 0 à (taille-1)

Tableaux à une et deux dimensions

Tab1.java

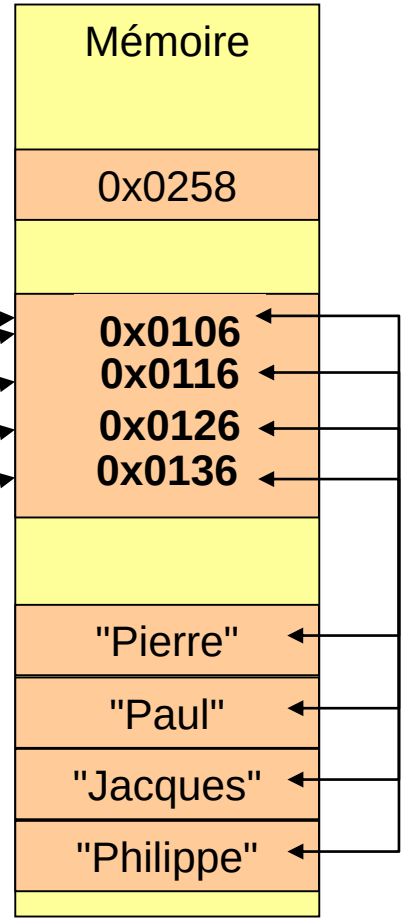
```
public class Tab1
{
  public static void main (String args[])
  {
    int tab[ ] ;
    tab = new int[4];
    tab[0]=5;
    tab[1]=3;
    tab[2]=7;
    tab[3]=tab[0]+tab[1];
  }
}
```



Tableaux à une et deux dimensions

Tab2.java

```
public class Tab2
{
  public static void main (String args[])
  {
    String tab[ ] ;
    tab = new String[4];
    tab[0]=new String("Pierre");
    tab[1]=new String("Paul");
    tab[2]=new String("Jacques");
    tab[3]=new String("Philippe");
  }
}
```



10. TABLEAUX

Tableaux à une et deux dimensions

```
// Tableau 1D
int[] notes = {12, 15, 8, 17, 10};
System.out.println(notes.length);   // 5
System.out.println(notes[0]);       // 12

double somme = 0;
for (int i = 0; i < notes.length; i++) {
    somme += notes[i];
}
double moyenne = somme / notes.length;

// Tableau 2D (matrice)
int[][] grille = new int[3][4];
grille[1][2] = 7;
```

Notions essentielles

- Taille fixe à la création : new int[5] ou initialisation directe { }
- Indices de 0 à length - 1 ; hors limites → ArrayIndexOutOfBoundsException
- tableau.length donne la taille (sans parenthèses, ce n'est pas une méthode)
- Tableau 2D = tableau de tableaux : int[][] grille = new int[lignes][colonnes]
- Les tableaux d'objets stockent des références (initialisées à null)

Méthodes et paramètres

```
public class Calculs {

    // méthode avec retour
    public static double moyenne(int[] notes) {
        double somme = 0;
        for (int n : notes) somme += n;
        return somme / notes.length;
    }

    // surcharge : même nom, signature différente
    public static double moyenne(int[] notes, double[] coeffs) {
        // ...
    }

    // méthode sans retour (void)
    public static void afficherMention(double moyenne) {
        System.out.println(moyenne >= 10 ? "Admis" : "Ajourné");
    }
}
```

Vocabulaire

- **Signature** = nom + type et nombre des paramètres (pas le type de retour)
- **Passage des paramètres par valeur** : une copie est transmise (y compris pour les références)
- **return** interrompt la méthode et renvoie une valeur (sauf void)
- **Surcharge** (**overload**) : plusieurs méthodes de même nom, signatures différentes
- **static** : méthode appelée sur la classe (Calculs.moyenne(...)), sans instance

- En java une méthode peut retourner une valeur et avoir une liste de paramètres :

  type_de_retour **nom_de_méthode** (*liste_des_paramètres*)
  {
  corps de la méthode
  }

- Exemple : Une méthode qui permet de retournerer la somme de deux entiers :

  int **addition**(*int a*, *int b*)
  {
  return a + b;
  }

11. MÉTHODES

TP 6 : Gestion des notes

- Reprends l'exercice de la gestion des notes (TP5), en écrivant un programme JAVA qui permet de gérer les notes de 30 étudiants au Max :
  - écrire une méthode ***saisie*** qui permet de saisir la note d'un seul étudiant ?
  - écrire une méthode ***moyenne*** qui permet de calculer la moyenne des notes ?
  - écrire une méthode ***trier*** qui permet de trier le tableau des notes dans un sens décroissant ?

```java
import java.util.Scanner;
public class GestionNotes {
    // Méthode pour saisir la note d'un seul étudiant
    public static double saisie() {  1 usage
        Scanner sc = new Scanner(System.in);
        double note;
        do {
            System.out.print("Saisir la note (0 à 20) : ");
            note = sc.nextDouble();
        } while (note < 0 || note > 20);
        return note;
    }
    // Méthode pour calculer la moyenne
    public static double moyenne(double[] notes, int n) {  1 usage
        double somme = 0;
        for (int i = 0; i < n; i++) {
            somme += notes[i];
        }
        return somme / n;
    }
```

11. MÉTHODES

```java
// Méthode pour trier les notes dans l'ordre décroissant
public static void trier(double[] notes, int n) {  1 usage
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (notes[i] < notes[j]) {
                double temp = notes[i];
                notes[i] = notes[j];
                notes[j] = temp;
            }
        }
    }
}
```

```java
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    final int MAX = 30;
    double[] notes = new double[MAX];
    int n;
    // Saisie du nombre d'étudiants
    do {
        System.out.print("Nombre d'étudiants (1 à 30) : ");
        n = sc.nextInt();
    } while (n < 1 || n > MAX);
    // Saisie des notes
    for (int i = 0; i < n; i++) {
        System.out.println("Étudiant " + (i + 1));
        notes[i] = saisie();
    }
    // Affichage de la moyenne
    System.out.println("\nMoyenne = " + moyenne(notes, n));
    // Tri décroissant
    trier(notes, n);
    // Affichage des notes triées
    System.out.println("\nNotes dans l'ordre décroissant :");
    for (int i = 0; i < n; i++) {
        System.out.println(notes[i]);
    }

    sc.close();
}
```

Organisation d'un projet Java/Maven

```
gestion-universitaire/
├── pom.xml                  <- config Maven
├── .gitignore
├── README.md
│
├── src/
│   ├── main/
│   │   ├── java/            <- code source
│   │   │   └── ma/uae/gestion/
│   │   │       └── Main.java
│   │   └── resources/       <- fichiers de config
│   │
│   └── test/
│       └── java/            <- tests JUnit
│
└── target/                  <- généré par Maven
```

Pourquoi Maven ?

- Structure standardisée : tout développeur Java s'y retrouve instantanément
- pom.xml décrit le projet : nom, version, dépendances, plugins
- Gestion automatique des dépendances (drivers JDBC, JUnit…)
- Cycle de build reproductible : compile, test, package
- target/ est généré automatiquement : ne jamais le versionner (.gitignore)

4. PROJET MAVEN

Cycle de build Maven

1

mvn validate

Vérifie que le projet est correct et que toutes les informations nécessaires sont disponibles.

2

mvn compile

Compile le code source du projet (src/main/java) vers target/classes.

3

mvn test

Exécute les tests unitaires (JUnit) via le framework de test approprié.

4

mvn package

Empaquette le code compilé dans son format distribuable, typiquement un .jar.

Chaque phase exécute automatiquement toutes les phases précédentes : "mvn package" compile et teste avant d'empaqueter. Commande de nettoyage : mvn clean supprime le dossier target/.

SYNTHÈSE

Synthèse Chapitre 1

- ✓ La JVM rend Java portable : bytecode compilé une fois, exécuté partout
- ✓ 8 types primitifs + types référence (String, tableaux, objets)
- ✓ Les tableaux ont une taille fixe, indexée à partir de 0
- ✓ Prochaine étape : structurer ce code avec la programmation orientée objet
- ✓ Un projet Maven suit une structure standard (src/main/java, pom.xml…)
- ✓ Structures de contrôle : if/switch, for/while/do-while
- ✓ Une méthode est identifiée par sa signature ; peut être surchargée

TP1 (S1) : mise en place JDK 21 + IntelliJ + Maven + Git · TP2 (S2) : mini-application console de gestion d'étudiants